Prg.no: Data Scaling using Min-Max Normalization Standardization

Aim:-

To apply and compare three different scaling techniques Min-Max normalization, Standardization and robust Scaling

Algorithm:-

* Import pandas, numpy and required scaling functions.
* Load the data into data frame.
* Select the numerical data
* Apply Min-Max Scaling.
* Apply standardization & robust Scaling using median.
* Store and compare result of three scaling.
* Use the appropriate scaled data for further analysis

Code:-

```
import matplotlib.pyplot as plt.
import numpy as np, pandas as pd, seaborn as sns
from sklearn.preprocessing import MinMaxScaler,
    robustScaler, StandardScaler.
df = pd.read_csv("expo4.csv")
print(df.head()); print("Initial Shape:", df.sh
print("Missing Values per column:\n", df.isnull().sum
df.columns = df.columns.str.strip().str.lower().st
    replace(" ", "_")
df["parental_level_of_education"] = df["parental_level_
  education"].fillna(df["parental_level_of_education"]
  mode()[0])
df = df.drop_duplicates()
```

```
print("Shape after removing duplicates:", df.shape)
numerical_cols = ["math_score", "reading_score", "writing_score"]
print(df[numerical_cols].head())
minmax = MinMaxScaler()
df_minmax = df.copy()
df_minmax[numerical_cols] = minmax.fit_transform
    (df[numerical_cols])
plt.figure(figsize=(15,4))
for i, col in enumerate(numerical_cols, 1):
    plt.subplot(1,3,i)
    sns.histplot(df_minmax[col], kde=True, color='green')
    plt.title(f"Min-Max Normalization - {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
standard_scaler = StandardScaler()
df_standard = df.copy()
df_standard[numerical_cols] = standard_scaler.fit_
    transform(df[numerical_cols])
plt.figure(figsize=(15,4))
for i, col in enumerate(numerical_cols, 1):
    plt.subplot(1,3,i)
    sns.histplot(df_standard[col], kde=True, color='blue')
    plt.title(f"Standard Scaled - {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
plt.tight_layout()
```

```
plt.show()
robust_scaler = RobustScaler()
df_robust = df.copy()
df_robust[numerical_cols] = robust_scaler.fit_transform
    (df[numerical_cols])
plt.figure(figsize=(15,4))
for i, col in enumerate(numerical_cols,1):
    plt.subplot(1,3,i)
    sns.histplot(df_robust[col], kde=True, color="orange")
    plt.title(f"Robust Scaled - {col}")
    plt.xlabel(col)
    plt.ylabel(frequency)
    plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

Output :-

Shape of dataset : (1000, 8) No of rows : 1000, No of col : 8

Duplicat row : 5

Missing Values :- Gender : 0
Race : 0
parent edu : 7
lunch : 0
Test Course : 0
Math Score : 0
Reading Score : 0
dtype : int64

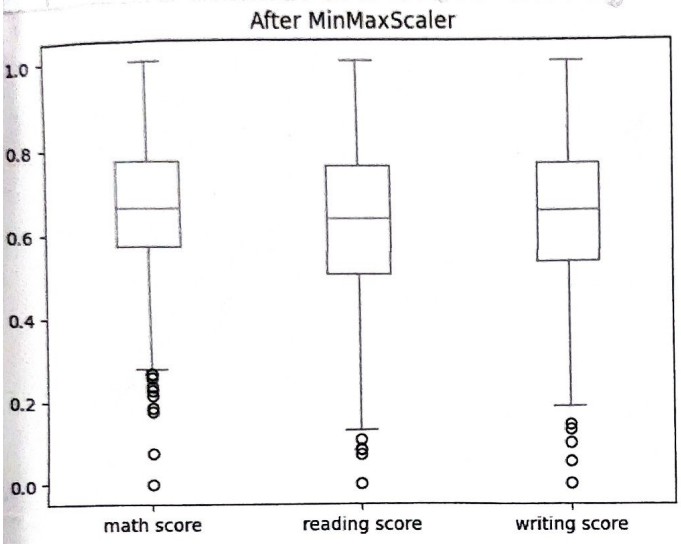
After Fillna : : 0
: 0
: 0
: 0
: 0
: 0
: 0
: 0

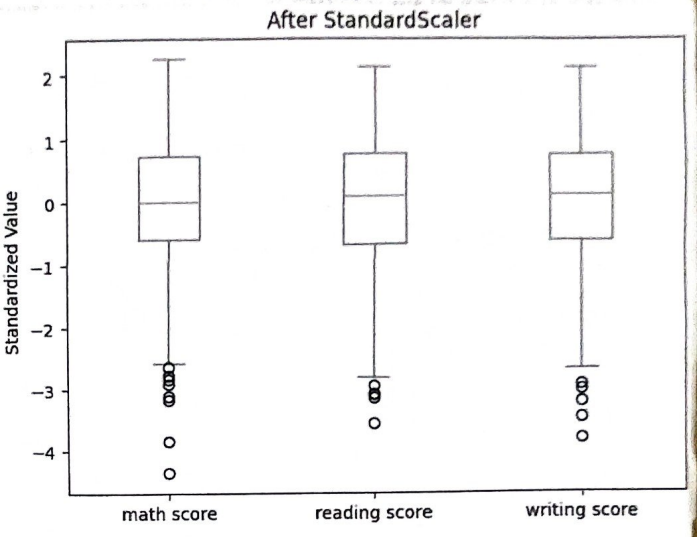
Original Encoding:

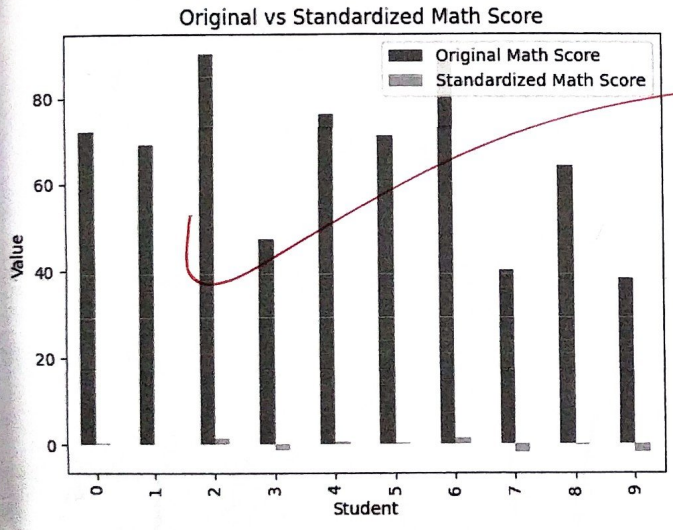
	Parent Education	Encoded
0	Bachelor's Degree	4.0
1	Some Clg	2.0
2	Master Degree	5.0
3	Associate Degree	3.0
4	Some clg	2.0

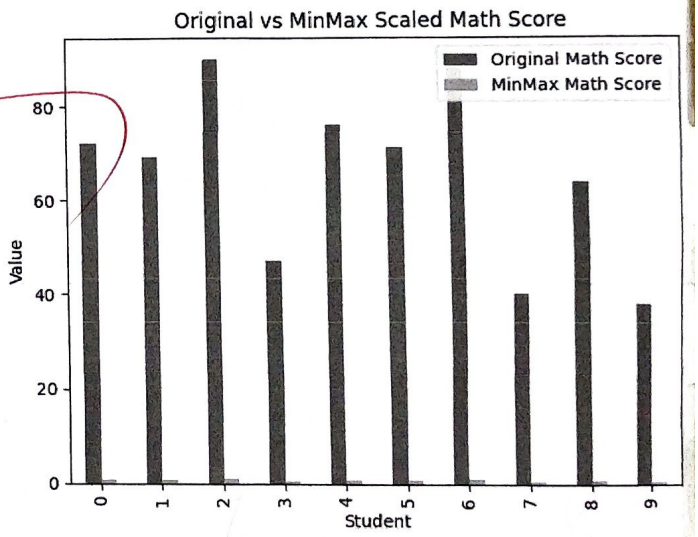
After Standard Scaler

	Math Score	Reading	Writing
0	0.387560	0.192711	0.390365
1	0.189745	1.425802	1.312135
2	1.574450	1.787305	1.641265
3	-1.260899	-0.834197	-1.584216
4	0.051313	0.603475	0.456395









Result:- 16/9/26

Thus the code for data scaling using Min Max normalization, Standardization and robust scaling was successfully executed.